

Calibrating the False-Positive Tax: Persona-Stratified Operator Cost of Autonomous AI Agent Observability

Krishna Chaitanya Balusu
Independent Researcher
krishnabkc15@gmail.com

Abstract—Observability stacks for autonomous AI agents are sold on a single number — accuracy, recall, or coverage — but the metric that decides whether on-call engineers can live with one is two-dimensional: the diagnostic quality when an engineer reaches for the trace, and the false-positive rate of the detectors that page her. We measure both on an open-source agent observability SDK using four disjoint corpora: a six-persona simulated diagnostic study (72 calls), a 112-trace, 3,060-span diagnostic-quality analysis, a real-LLM false-positive evaluation, and a threshold-sensitivity sweep over three knobs. Agent-specific span kinds lift persona-averaged accuracy 25.0%→91.7%, but the lift inverts seniority: QA engineers and tech leads reach 100%; site-reliability engineers, already stack-trace-fluent, gain least. Two of three detector knobs reshape operator load smoothly; the third trades visible false-positives for silent false-negative risk past a threshold most teams will cross. Pick the portfolio by the persona who carries the pager.

Index Terms—Human-AI collaboration, AI agent observability, alert fatigue, false-positive rate, threshold calibration, on-call engineering, diagnostic quality, OpenTelemetry, software engineering, empirical study.

Three things an on-call team can do on Monday

- **Staff the rotation by who gains, not by seniority.** Agent-specific span kinds lift diagnostic accuracy from ~17% to 100% for QA engineers and tech leads, but only from 67% to 83% for site-reliability engineers — who already read stack traces fluently. The cheapest sustainable rotation puts the engineers with the steepest lift on-call, not the most senior.
- **Before raising any noise-suppression threshold, run a five-point sensitivity sweep.** Two of three common detector knobs reshape operator load smoothly; the third (`token_growth_factor`) silently zeros an entire alert class past 1.45 — substituting visible false-positives for invisible false-negatives. Sweep first; tune second.

Transparency disclosures (first page, by author choice). (1) **Prior author work.** The cited AgentTelemetry SDK paper [1] is by the same author; the present article uses that SDK as the measurement instrument and does not replicate the prior paper’s failure-detection-coverage claim — it measures a different property (the operator cost of using the SDK). (2) **Concurrent submission.** The author is concurrently submitting to a different IEEE Software Special Issue (The Edge-Cloud Continuum, deadline 7 July 2026) on a disjoint research question (cross-tier replication of a published intervention on SWE-bench Lite outcome runs) with no shared corpus and no manuscript overlap; an orthogonality artifact is on file with the present submission.

- **Triage your real-issue backlog into “telemetry-reachable” and “narrative-only.”** On 17 production agent bugs across five frameworks, 35% had no log evidence any detector could act on. Tag those tickets at intake so the on-call engineer routes them to the model owner immediately — not after the detector silently fails to fire.

I. THE PAGER AT 3 A.M.

A junior engineer, new to the on-call rotation, is paged at 3 a.m. because an autonomous coding-assistant agent has crossed a “potential reasoning loop” threshold on a Django backport (instance `django__django-10914`, a `FILE_UPLOAD_PERMISSION` configuration bug). She has 90 seconds before the customer-facing time-to-respond clock turns yellow.

On her vendor’s vanilla trace view, she sees a flat list of six failed `search_code` calls, each returning empty. She files this as “tool error — search index stale,” silences the page, and goes back to bed. The cost of that misdiagnosis is paid the next shift: the search calls were fine, but the agent was looping through reasoning steps that the vanilla view never rendered as spans. She had triaged the symptom and missed the fault. In our six-persona simulation on this exact instance, every single persona reaches the same wrong diagnosis under vanilla telemetry — and every persona reaches the correct “reasoning loop” diagnosis when the same trace is rendered with agent-specific span kinds (see Section III).

That 90 seconds is two-dimensional. There is the diagnostic-quality axis: when she looks, do the spans tell her what happened? And there is the false-positive-rate (FPR) axis: was this page right to wake her up? A stack that scores well on one axis and badly on the other does not survive contact with a 24-by-7 rotation. Observability vendors sell into that 90-second window with a single headline number — “94% recall on common agent failure modes,” say. The on-call engineer’s experience is what the headline number elides.

The recent literature on human-AI collaboration in software engineering has converged on iterative, partnered workflows as the right human-centered pattern [3], [4]. What that literature has not yet measured is the upstream input that decides whether such iteration is even reachable: the operator cost the underlying observability stack imposes before the human can

begin to collaborate. This article quantifies that cost, using an open-source agent observability SDK [1] as the instrument.

Two findings reshape the operator-cost conversation. **First**, agent-specific span kinds lift persona-averaged diagnostic accuracy from 25.0% to 91.7% — but the lift is sharply uneven by role, and the engineers who gain most are not the ones currently staffing most on-call rotations. **Second**, three commonly-cited detector calibration knobs are not interchangeable: two reshape operator load smoothly, while the third quietly substitutes false-positive cost for silent false-negative risk. Three concrete moves an on-call team can make on Monday follow.

II. TWO AXES OF OPERATOR COST

Diagnostic quality is the operator’s decision-accuracy when paged. It is what determines whether a 90-second triage produces a correct “silence,” “page the model owner,” or “page the customer-facing team” decision. We measure it both at the persona level (how often each engineering role reaches the correct root-cause classification) and at the span level (how many spans the operator examines, and how localized the diagnostic signal is).

False-positive rate is the fraction of pages that should never have fired. Every false-positive page is a *tax* on operator attention — a withdrawal from a finite trust budget that, when overdrawn, ends on-call careers. We call the cumulative withdrawal the *false-positive tax*: the operator-attention cost a deployed detector portfolio imposes per unit time on the engineers carrying its pages. The trust-calibration literature has long-established that recurring false alarms erode warranted human reliance on automation [7], and the site-reliability literature names alert fatigue as the single most cited cause of on-call rotation attrition [8].

These two axes interact. A low-FPR stack with poor diagnostic quality produces rare pages the operator cannot triage. A high-diagnostic-quality stack with high FPR produces clear answers to questions the operator should not have been forced to ask. The operationally-viable stack is the one calibrated on both axes against the persona who carries the pager.

The AgentTelemetry SDK [1] introduces nine agent-specific span kinds (AGENT, LLM_CALL, TOOL_CALL, PLANNING, REASONING, RETRIEVAL, GUARD_RAIL, DELEGATION, MEMORY) extending the OpenTelemetry GenAI conventions [2]; we treat the SDK as the instrument under measurement, not as the contribution. The same SDK was independently validated against the MAST taxonomy [5] of multi-agent failure modes. The prior work introduces the SDK and quantifies its failure-detection coverage on a fixed benchmark; it does not measure the persona-stratified diagnostic accuracy, threshold-sensitivity calibration curve, or operator-cost properties we characterize here.

III. PERSONA-STRATIFIED DIAGNOSTIC QUALITY

The least intuitive number in this study is that site-reliability engineers gain the least from agent-specific instrumentation. They are the engineers most often asked to staff agent on-call

rotations, and they are the persona for whom the instrumentation lift is smallest. That inversion — and what it implies for who should actually be on the rotation — is the human-centric finding of the persona study.

We ran six engineering personas — Junior SWE, Senior Backend, ML Engineer, SRE/DevOps, QA Engineer, Tech Lead — each prompted as a developer reviewing failed-agent traces (not as the on-call rotator in the lead vignette, who is downstream of those reviews). *The personas are LLM-simulated* (GPT-4o-mini playing each role under role-specific prompts) rather than human developers; this is the load-bearing methodological choice of the study and we return to its limits in the closing section. Each persona diagnosed the same six SWE-bench Lite [6] failed-agent traces under two conditions: with the nine AgentTelemetry agent-specific span kinds visible, and with only Vanilla OpenTelemetry GenAI spans visible. Each persona issued 12 diagnoses (6 traces $\times$ 2 conditions), for 72 total LLM-judged diagnoses. Each diagnosis returns a structured root-cause label, the number of spans examined, and a HIGH/MED/LOW confidence rating.

Aggregate accuracy is 91.7% with agent-specific spans versus 25.0% with Vanilla OTel (Wilson 95% confidence intervals [78.2, 97.1] vs. [13.8, 41.1], $n=36$ per condition) — a +66.7pp lift in correct root-cause identification. The span-examination cost is modest: 23.1 spans examined on average with agent-specific spans versus 19.2 with Vanilla OTel, a 20.4% increase. Operators trade roughly one extra screenful of spans for nearly three times the diagnostic accuracy.

The per-persona breakdown (Table I) is where the inversion shows. QA Engineer and Tech Lead reach 100.0% accuracy with agent-specific spans, against 16.7% with Vanilla OTel — the largest deltas, +83.3pp. Junior SWE, Senior Backend, and ML Engineer all reach 83.3%–100% with agent-specific spans against 16.7% baseline. SRE/DevOps is the outlier in both directions: highest Vanilla-OTel accuracy (66.7%), smallest lift (+16.7pp). SRE/DevOps engineers are already trained to reason from minimal stack-trace evidence; their baseline is high because they are practiced at reading the signal even when it is not there. The other personas are not.

The operational consequence is direct: *the cheapest sustainable rotation is not the one with the most senior engineers; it is the one staffed by the personas with the steepest lift*. A QA-engineer-staffed rotation on agent-specific telemetry reaches the same diagnostic accuracy as an SRE-staffed rotation on vanilla telemetry, at a fraction of the headcount cost — and frees the SREs for the genuinely-hard cases the detectors will not surface.

The diagnostic-quality finding is corroborated at the span level by an independent 112-trace, 3,060-span analysis [1] that we report only briefly here: localization precision (number of spans an operator examines to identify the failing primitive) drops from a mean of 9.49 with Vanilla OTel to 3.00 with agent-specific spans, a 68.4% reduction; spans-to-diagnosis drops from 28.55 to 9.46, a 66.8% reduction. These span-level numbers come from a different instance corpus than the persona study, and they cross-validate the persona-level signal.

TABLE I

PERSONA-STRATIFIED DIAGNOSTIC ACCURACY ON 6 SWE-BENCH LITE [6] FAILED-AGENT TRACES. EACH PERSONA ISSUED 12 DIAGNOSES (6 TRACES $\times$ 2 CONDITIONS) FOR 72 LLM-JUDGED DIAGNOSES TOTAL (SIMULATED_USER_STUDY / [12]). CONFIDENCE: HIGH ON ALL 72 DIAGNOSES BY SELF-REPORT.

Persona	Role context	AT acc.	OTel acc.	Δ (pp)	AT spans
Junior SWE	Generalist, <2y experience	83.3%	16.7%	+66.6	21.3
Sr. Backend	Application-layer specialist	100.0%	16.7%	+83.3	18.0
ML Engineer	Model-side specialist	83.3%	16.7%	+66.6	26.3
SRE/DevOps	Stack-trace fluent, on-call	83.3%	66.7%	+16.6	19.7
QA Engineer	Test-oriented, less trace exposure	100.0%	16.7%	+83.3	27.3
Tech Lead	Owns rotation, audits diagnoses	100.0%	16.7%	+83.3	26.2
Average	—	91.7%	25.0%	+66.7	23.1

Wilson 95% confidence intervals: AT [78.2, 97.1], OTel [13.8, 41.1]. Average spans examined under Vanilla OTel: 19.2 (95% CI [17.3, 21.3]). The SRE/DevOps row is the outlier; the other five personas all show +66pp or larger lifts.

IV. THE FALSE-POSITIVE TAX

False-positive rate matters because every false page is a withdrawal from the on-call engineer’s trust budget. We measured the four canonical AgentTelemetry detectors — infinite-retry, cost-explosion, token-growth (context overflow), hallucination-candidate — on 112 real-LLM traces comprising 3,060 spans (real_fpr/fpr_results.json). At the published default thresholds (max_retries=5, cost_threshold=0.5 USD, token_growth_factor=2.0, hallucination_min_confidence=0.5), the combined FPR is 0.0%: zero false alarms on 112 successful agent runs.

That clean cell is misleading on its own. The relevant operator question is not “what is the FPR at the recommended threshold?” but “how does the FPR move when I tune the threshold for my deployment?” Table II reports the threshold-sensitivity sweep across three commonly-tuned detector knobs at five settings each, on two trace corpora totaling 10,009 spans (threshold_sensitivity / [12]).

The headline finding from Table II is that the three knobs are not interchangeable for operator-cost calibration.

max_retries reshapes operator load *smoothly*: lowering from 3 to 2 raises alert volume from 26 to 34 on the real-LLM corpus (+30.8%); raising past 4 saturates at near-zero alerts. The operator can position themselves anywhere on this curve depending on rotation size and SLA tolerance.

cost_threshold is essentially flat across \$0.05–\$0.15: alert count is invariant at 26 on the real-LLM corpus. Outside the tested range, extreme values would page on every run or never; within the operator-relevant range, this knob is not the lever for managing alert volume.

token_growth_factor is the dangerous knob. At 1.15 it produces 59 alerts on the real-LLM corpus, +127% over default — the kind of false-positive flood that motivates an engineering manager to “just raise the threshold.” But raising to 1.45 or higher does not merely reduce false positives; it silently drops the entire context-overflow detector to zero alerts. The detector’s threshold has exceeded any observed token-growth on the corpus; the alerts that previously fired (some of which were genuine) now do not fire at all. The operator who raised the threshold to silence noise has traded

a false-positive cost they could see for a false-negative risk they cannot.

The human-centric implication is direct. The naive engineering-management instinct — “the on-call rotation is complaining about noise, raise the thresholds” — works smoothly on max_retries and is harmless on cost_threshold, but on token_growth_factor it converts a visible operator cost into an invisible operational risk. *Calibration is not a single knob; it is a portfolio decision, and the three knobs are not equivalent.*

V. CROSS-TOOL HEAD-TO-HEAD

We compared the operator-visible signal that three observability stacks expose on the same 112-trace failure-mode corpus: Vanilla OpenTelemetry, OpenLLMetry [9], and AgentTelemetry (Table III, source head_to_head/comparison_table.tsv). Vanilla OTel exposes zero agent-specific span kinds; OpenLLMetry exposes four (covering tool errors, LLM cost, and four span kinds); AgentTelemetry exposes nine, including the planning, reasoning, guardrail, and memory phases that produce most of the operator-relevant alert traffic.

For the on-call engineer, the operationally salient cells are the 72 reasoning-loop events and the 34 guardrail events — both invisible to Vanilla OTel and OpenLLMetry. These are precisely the events that, on the persona-study traces, the QA Engineer and Tech Lead personas relied on to reach 100% diagnostic accuracy. A team running on Vanilla OTel can buy a noise-free pager (no detector can fire because no signal is exposed), but cannot diagnose the events when the model fails. A team running on OpenLLMetry sees cost overruns and tool errors but is blind to the reasoning-loop class of fault.

VI. THE 35% TELEMETRY CANNOT REACH

We checked detector applicability against 17 real GitHub-issue agent faults from LangChain [10], LangGraph, langchain-aws, CrewAI [11], and NVIDIA NeMo Guardrails, spanning six fault types. 11 of 17 (64.7%) had reconstructible telemetry signal that maps onto a detector trigger; 6 of 17 (35.3%) did not — they were narrative-only memory or retrieval misbehaviors. langchain#3354 (stale-retrieval) is reconstructible from issue text; crewAI#827 (short-term

TABLE II
THRESHOLD-SENSITIVITY SWEEP. ALERT COUNTS WHEN EACH OF THREE DETECTOR KNOBS IS VARIED ACROSS FIVE SETTINGS ON TWO TRACE CORPORA: SWE-BENCH (3,060 SPANS) AND REAL-LLM (6,949 SPANS). DEFAULT SETTINGS ARE MARKED $\star$.

Knob	Setting	SWE-bench	Real-LLM	Effect on operator load
max_retries	2	15	34	Page rate roughly doubles (real-LLM +30.8%)
	3 $\star$	7	26	Baseline
	4	5	8	Page rate cut sharply (real-LLM -69.2%)
	5	3	6	Saturating
	6	3	6	Saturated
cost_threshold	0.05	7	26	Insensitive across tested range
	0.075	7	26	
	0.1 $\star$	7	26	Baseline
	0.125	7	26	
	0.15	7	26	
token_growth_factor	1.15	25	59	Page rate +127% on real-LLM (false-positive spike)
	1.3 $\star$	7	26	Baseline
	1.45	4	21	Context-overflow alerts drop to zero
	1.6	4	21	Context-overflow alerts still zero
	2.0	4	21	Context-overflow alerts still zero

max_retries and token_growth_factor change alert volume sharply; cost_threshold is insensitive in this range. Critically, raising token_growth_factor past 1.45 silently disables the context-overflow detector — a false-positive reduction substituted for a false-negative risk. *Per-detector decomposition at the default token_growth_factor=1.3 on the real-LLM corpus:* 21 of the 26 baseline alerts come from the infinite-retry detector and 5 from context-overflow; at 1.45 the infinite-retry contribution is unchanged at 21, but the context-overflow contribution drops to 0. The 5-alert decrease is the entire context-overflow class disappearing, not a partial denoising. Poisson 95% confidence intervals on the real-LLM counts: 26 alerts has CI [17.0, 38.1]; 59 alerts has CI [44.9, 76.1]; 21 alerts has CI [13.0, 32.1]. The 1.15 $\rightarrow$ 1.45 excursion is significant; the 1.45 $\rightarrow$ 2.0 saturation is not.

TABLE III
OPERATOR-VISIBLE SIGNAL ACROSS THREE AGENT OBSERVABILITY STACKS ON THE SAME 112-TRACE FAILURE-MODE CORPUS.

Operator-relevant signal	V. OTel	OpenLLM.	AT
Visible agent span kinds	0	4	9
Reasoning-loop events surfaced	0	0	72
Guardrail events visible	0	0	34
Planning-phase events visible	0	0	112
Memory operations visible	0	0	224
Tool-error events detected	0	17	17
LLM cost tracked	no	yes	yes
Trace data lost	100%	42%	0%

memory loss) is a narrative complaint with no log evidence any detector could act on.

The actionable move is at intake, not at runtime: tag tickets “telemetry-reachable” or “narrative-only” as they land so the on-call engineer routes the narrative-only third to the model owner immediately, rather than waiting for a detector that will silently fail to fire.

VII. WHAT WE DON’T KNOW YET

Simulated personas, not human developers. The 72 diagnostic calls in Section III were issued by an LLM (GPT-4o-mini) prompted to play each of the six persona roles. The finding is consistent with the literature on LLM-as-judge for SE tasks but does not substitute for an IRB-approved study with human operators. Mapping the full design space of detector portfolio $\times$ persona $\times$ trace requires hundreds of diagnoses, which is impractical with human subjects in any single study; paired small- n human plus large- n simulated studies are the natural next step.

Single-benchmark scope for diagnostic quality. The persona-study traces are drawn from six SWE-bench Lite [6] failed-agent runs (one each from Django, SymPy, Matplotlib, scikit-learn, pytest, Sphinx). Coding-agent faults are a domain-specific subset of agent faults; document-QA, multi-agent orchestration, and tool-use benchmarks could each shift the persona-stratified pattern. The threshold-sensitivity corpus, by contrast, mixes SWE-bench and real-LLM traces and is genuinely cross-domain.

FPR=0.0% at a single threshold cell. The 0.0% combined FPR on 112 traces is a clean result at one threshold setting; the sensitivity sweep documents that the FPR landscape is non-trivial around that setting. We report the 0.0% headline because it is the published default; we report the sensitivity sweep because the published default is not where every team will sit.

Detector-applicability sample size. The 17-issue applicability sample (Section VI) is hand-coded by the author and small. It establishes that some non-trivial fraction of real-world faults is non-reconstructible; the precise rate (35.3% in this sample) should not be over-quantified.

VIII. CONCLUSION

The junior engineer at 3 a.m. does not need a higher-recall detector. She needs a stack that pages her when she should be paged, and that shows her what happened when she looks. Both axes — diagnostic quality and false-positive rate — are persona-sensitive and threshold-sensitive in ways that the single-number marketing of agent observability obscures.

Two findings reshape the operator-cost conversation: agent-specific span kinds broaden which engineering roles can carry the pager (and the roles that gain most are not the ones

currently staffing most rotations); and one of three commonly-tuned calibration knobs silently substitutes false-positive cost for false-negative risk past a threshold most teams will cross on the way to silencing noise. The three actionable insights boxed at the front of this article are the practical reduction.

The next agent-observability stack you evaluate should publish persona-stratified diagnostic accuracy, not just headline recall — and should distinguish which of its detector knobs reshape operator load from which substitute one operator risk for another. Iterative human-agent collaboration of the kind the recent literature [3], [4] converges on is the right human-centered pattern; the operator-cost calibration described here is the precondition that makes it reachable.

Choose the portfolio for the engineer who carries the pager.

REPRODUCIBILITY

The persona-study transcripts, the threshold-sensitivity sweep, the head-to-head comparison, the detector-applicability annotations, and the underlying SDK source are available in the AgentTelemetry repository [12]. Supplementary data is archived in IEEE DataPort post-acceptance.

ACKNOWLEDGMENTS

(Camera-ready only.)

REFERENCES

- [1] K. C. Balusu, “AgentTelemetry: A fault detection benchmark and toolkit for LLM agent observability,” in *Proc. 3rd ACM Int’l Conf. on AI-Powered Software (AIware ’26)*, Benchmark & Dataset Track, Montreal, QC, Canada, Jul. 6–7, 2026. DOI: 10.1145/3805760.3814931.
- [2] OpenTelemetry Authors, “Semantic conventions for generative AI systems,” *OpenTelemetry Specification*, 2026. [Online; accessed 2026-05-17]. <https://opentelemetry.io/docs/specs/semconv/gen-ai/>.
- [3] A. Kumar, Y. Bajpai, S. Gulwani, G. Soares, and E. Murphy-Hill, “Why AI agents still need you: Findings from developer-agent collaborations in the IDE,” in *Proc. 40th IEEE/ACM Int’l Conf. on Automated Software Engineering (ASE ’25)*, 2025. arXiv:2506.12347.
- [4] S. Abrahão, et al., “Software engineering by and for humans in an AI era,” *ACM Trans. Softw. Eng. Methodol.*, 2025.
- [5] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, “Why do multi-agent LLM systems fail?” arXiv:2503.13657, 2025.
- [6] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. ICLR*, 2024.
- [7] K. Okamura and S. Yamada, “Adaptive trust calibration for human-AI collaboration,” *PLOS ONE*, vol. 15, no. 2, e0229132, 2020.
- [8] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy (eds.), *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly, 2016. Chapter on alert design and operator fatigue.
- [9] Traceloop, “OpenLLMetry: Open-source observability for LLM applications,” 2025. [Online; accessed 2026-05-17]. <https://github.com/traceloop/openllmetry>.
- [10] H. Chase et al., “LangChain: Building applications with LLMs through composability,” 2023–2026. [Online; accessed 2026-05-17]. <https://github.com/langchain-ai/langchain>.
- [11] J. Moura et al., “CrewAI: Framework for orchestrating role-playing autonomous AI agents,” 2024–2026. [Online; accessed 2026-05-17]. <https://github.com/crewAIInc/crewAI>.
- [12] K. C. Balusu, “AgentTelemetry: Source code, experimental corpora, and analysis scripts,” 2026. (Repository URL withheld pending peer review; will be linked in camera-ready.)

AUTHOR BIOGRAPHY

Krishna Chaitanya Balusu is an independent researcher focusing on distributed systems reliability engineering and observability for autonomous AI agents. He is the author of the AgentTelemetry observability SDK (open-source, 2026) and his research interests include operator-cost characterization of telemetry portfolios and reproducibility methodology for empirical systems studies. He holds an M.S. in Computer Science from the Georgia Institute of Technology and is a Senior Member of IEEE. Contact him at krishnabkc15@gmail.com.